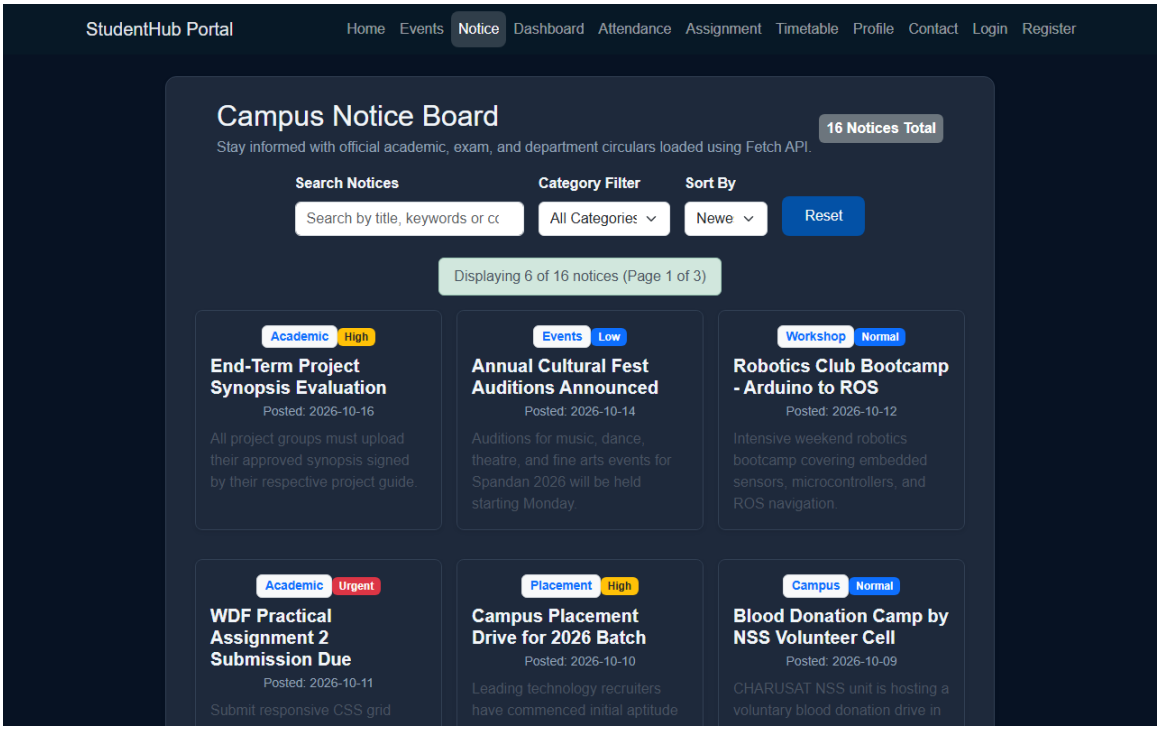
PRACTICAL – 6

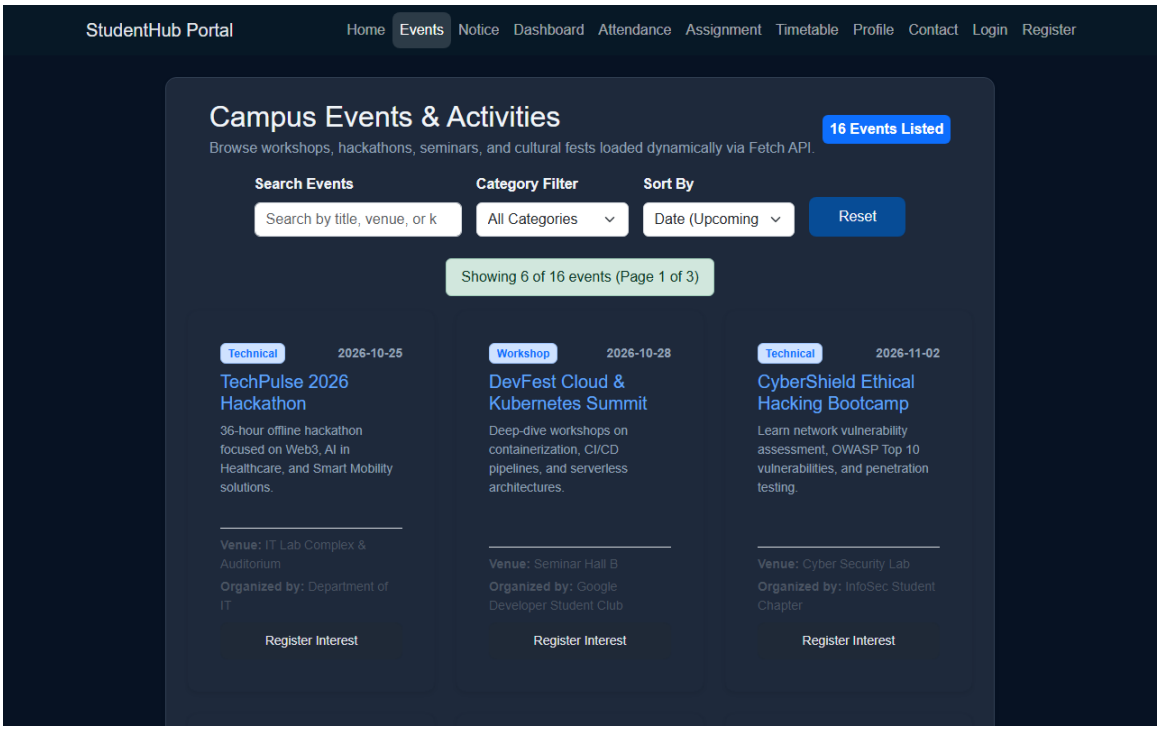
Problem Definition: Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination.

Output:

Notice Board Rendered Dynamically via Fetch API with Category Filter:



Campus Events Directory with Search, Filter & Pagination:



Question & Answer:

1. How is JSON fetched, parsed, and rendered?

Ans. External JSON files (data/notices.json and data/events.json) are fetched asynchronously using the JavaScript Fetch API. The response promise is parsed into JavaScript objects using response.json(). The parsed data array is then iterated using the map() method to generate HTML card markup, which is inserted into the DOM container using innerHTML.

2. Which array methods are used for search, filter, sort, and pagination?

Ans. Array filter() is used to match search terms and category criteria, sort() is used to arrange records by date (newest/oldest) or title alphabetically, and slice() is used to extract the exact slice of records corresponding to the current active page for client-side pagination.

3. How are loading and error states handled?

Ans. Before data arrives, an informative loading state with ARIA attributes is displayed. If network requests fail, catch() blocks display user-friendly error banners and retrieve previously cached records from localStorage as an offline fallback.

4. How is modularity maintained in JavaScript files?

Ans. JavaScript functions are separated into discrete, well-defined modules (initializeNotices(), initializeEventsDirectory(), renderNotices(), populateCategories()). Each module manages its own event listeners, state variables, and rendering logic, ensuring clean separation of concerns and maintainability.

Conclusion / Learning Outcome:

This practical successfully implemented dynamic data consumption from external JSON files (events.json, notices.json, students.json, faqs.json) using modern JavaScript Fetch API. Features including real-time search, category filtering, multi-criteria sorting, client-side pagination, and offline localStorage fallback caching were built and validated.

Github link :- https://github.com/07vish/sh.git